

Analise - Arquitetura de Provisionamento de Ramais Asterisk

1. Recomendacao principal

- Usar um agente de provisionamento restrito na VPS, acessado por API autenticada pelo LigFlow. E a melhor opcao para o estado atual: mantem o Asterisk isolado, evita SSH no PHP e preserva NVOIP_DIRECT, ARI e a discagem existente.

2. Arquitetura proposta

- LigFlow cria uma reserva de ramal e um job idempotente no banco. Um agente local da VPS recebe apenas operacoes permitidas (create, update, disable), valida os dados, gera arquivos dedicados do LigFlow em pjsip.d, faz backup, executa somente reload PJSIP permitido e confirma o endpoint.

- ARI continua apenas para operacao, eventos e diagnostico. AMI nao e necessario para provisionar ramais.

3. Fluxo de criacao

- Admin cria usuario e escolhe ramal.
- LigFlow valida e reserva o ramal em transacao (BEGIN IMMEDIATE).
- Gera senha SIP forte no backend e armazena criptografada.
- Cria job de provisionamento com chave de idempotencia.
- Agente cria endpoint, auth e aor PJSIP no include dedicado.
- Agente valida, recarrega PJSIP e confirma o endpoint.
- LigFlow marca o vinculo como ativo/provisionado.
- O sip_config do usuario passa a usar seu proprio ramal e senha.

4. Falha e rollback

- Se o agente falhar, o usuario permanece criado, mas o vinculo fica FAILED ou pendente e nao e usado pelo WebRTC nem pela resolucao ARI. O job pode ser repetido sem duplicar endpoint.
- Se o Asterisk aplicar e a confirmacao no LigFlow falhar, um reconciliador consulta o resultado pelo idempotency_key antes de qualquer compensacao. Nunca apagar automaticamente um ramal sem confirmar o estado remoto.

5. Banco

- Reutilizar a tabela asterisk_user_extensions e ampliar, em vez de criar outra tabela de vinculo.
- Campos recomendados: sip_password_encrypted; lifecycle_status (RESERVED, ACTIVE, FAILED, RELEASING, RELEASED); provisioned_at; last_provision_error; provisioning_version ou config_fingerprint; released_at.
- Criar uma tabela pequena de asterisk_provisioning_jobs para outbox, tentativas, idempotencia e auditoria do resultado remoto.
- O atual asterisk_server_id = 1 pode continuar nesta fase, mas deve deixar de ser espalhado ou hardcoded. Hoje ha uma unica configuracao global em asterisk_settings; quando houver mais servidores, ela deve evoluir para uma entidade de servidores.

6. Impacto no código atual

- O ponto de vínculo já existe em `sync_user_asterisk_extension()`, e a resolução por eventos ARI já ocorre no processamento de eventos Asterisk.
- A mudança futura deve restringir a resolução automática a ramais ACTIVE e PROVISIONED. O endpoint WebRTC atual e a senha global em `asterisk_settings` deixam de ser a fonte para usuários provisionados, sem alterar o fluxo NVOIP_DIRECT.

7. Riscos

- Arquivo PJSIP manual e arquivo gerenciado não podem definir o mesmo ramal.
- Senha SIP precisa aparecer somente ao navegador autenticado do próprio usuário, nunca em listagens, logs ou auditoria.
- SQLite exige reserva transacional para evitar dois usuários obterem o mesmo ramal.
- Realtime Asterisk é mais escalável, mas hoje acrescentaria ODBC, banco compartilhado e risco operacional desnecessário.
- Alterar `pjsip.conf` diretamente pelo LigFlow ou por SSH é o caminho menos seguro e não é recomendado.

8. Etapas pequenas

- Definir ciclo de vida do vínculo e criar outbox de provisionamento.
- Criar agente da VPS com API restrita, HMAC ou mTLS, allowlist de IP e operações idempotentes.
- Homologar criação, atualização e desativação de um ramal por include dedicado.
- Integrar o job do LigFlow ao agente e exibir estado e retry administrativo.
- Ajustar `sip_config` para credencial individual provisionada.
- Adicionar reconciliação, auditoria e liberação segura do ramal.
- Só então avaliar Asterisk Realtime para escala maior.

Confirmação

- Nenhuma alteração foi feita durante a análise.